

STARK-QA: A Reproducible Hybrid RAG–KAG System for Narrative Question Answering on FairytaleQA

Ishaan Romil (2023114011) Archit Choudhary (2023114002) Sujal Deoda (2022115001)
International Institute of Information Technology, Hyderabad

Abstract

We present **STARK-QA** (*Story-grounded Text Answering with Retrieval and Knowledge*), a question-answering system for the FairytaleQA narrative comprehension benchmark that fuses dense + sparse passage retrieval (RAG) with a lightweight knowledge graph (KAG) and generates grounded answers with a large language model. Beyond the system itself, this report documents a *reproducibility and repair* effort: we took a non-running research prototype, restored it to a fully working state, hardened every external dependency with a graceful fallback, and pinned a locked, one-command environment. On 100 held-out FairytaleQA test questions, STARK-QA attains a token-level F1 of **0.611**, **3.5**× the Basic-RAG ablation (0.176) and **14.9**× the Basic-KAG ablation (0.041), while leading on Exact Match, ROUGE-1/2/L, BLEU, semantic similarity and BERTScore. We release the full pipeline, cached data splits, and a deterministic uv lockfile.

1 Introduction

Narrative question answering requires a system to read a story and answer free-form questions about its characters, settings, actions, and the causal and emotional relationships between them. FairytaleQA (Xu et al., 2022) is an authentic benchmark of such questions written by education experts over classic fairy tales, spanning seven attribute types (*character, setting, action, feeling, causal relationship, outcome resolution, and prediction*) and an explicit/implicit reasoning split.

Two paradigms dominate knowledge-intensive QA. Retrieval-augmented generation (RAG) (Lewis et al., 2020) retrieves relevant passages and conditions a generator on them; it excels at descriptive *where/why* questions whose answer is a span of text. Knowledge-augmented generation (KAG) instead extracts structured entities and relations (Pan et al., 2024) and is naturally suited

to entity-centric *who* questions. STARK-QA **fuses both**: it orders RAG and KAG evidence by the question type and lets an LLM compose a short, grounded answer, so it inherits the strengths of each rather than committing to one.

This report makes three contributions:

1. A hybrid RAG+KAG architecture with question-type-aware evidence fusion and grounded LLM generation (§2).
2. A reproducibility/repair effort that turns a broken prototype into a one-command, dependency-locked, gracefully degrading pipeline (§3).
3. A full evaluation against RAG-only and KAG-only ablations across eight metrics and seven question types (§5).

2 System Architecture

Given a question q and the story corpus reconstructed from the test split, STARK-QA produces an answer in four stages.

Hybrid retrieval (RAG). Stories are chunked into overlapping word windows (200 words, overlap 40). We embed chunks with a SentenceBERT encoder (Reimers and Gurevych, 2019) (all-MiniLM-L6-v2) and index them with FAISS (Johnson et al., 2021) for dense semantic search, run in parallel with a TF-IDF lexical index. The two score vectors are min-max normalised and fused as $s = \alpha s_{\text{dense}} + (1 - \alpha) s_{\text{tfidf}}$ with $\alpha = 0.6$, followed by a light question-type re-ranking pass.

Knowledge graph (KAG). From each story we extract entities (characters, objects/places, descriptive concepts), simple co-occurrence relationships, and the sentences in which each entity appears, all with dependency-free regular expressions. At query time the question is matched to entities (exact $\rightarrow$ partial $\rightarrow$ semantic) and the most relevant

stored sentences plus matching relationships are returned.

Question-type-aware fusion. RAG and KAG evidence are ordered by the kind of question—entity-first for *who*, passage-first for *where/why*—before generation, so the prompt foregrounds the most useful evidence for that type.

Grounded generation. A few-shot, FairytaleQA-styled prompt conditions the Mistral LLM (`mistral-small-latest`, temperature 0) on the fused evidence to produce the shortest complete answer span. When no API key is present, the system falls back to a dependency-free extractive answerer over the same fused evidence, so it always returns a grounded answer.

Ablation baselines. **Basic-RAG** uses TF-IDF retrieval with purely extractive answering (no dense embeddings, no KAG). **Basic-KAG** builds an entity co-occurrence graph and answers from graph neighbourhoods. Both are deliberately weaker, isolating the contribution of hybrid retrieval and of the fusion-plus-LLM design.

3 Reproducibility and Engineering

A substantial part of this work was restoring a non-running prototype to a correct, reproducible state. The salient changes were:

- **Shared utilities.** Sentence splitting, tokenisation, answer normalisation and corpus fingerprinting were centralised, removing behavioural drift between components that previously defined these differently.
- **Graceful degradation.** Every heavy dependency (`torch/sentence-transformers/FAISS`, `spaCy`, `NLTK`, `ROUGE`, `BERTScore`, `Matplotlib`) has a pure-Python or TF-IDF fallback, so a missing package degrades quality rather than crashing the run.
- **LLM client compatibility.** Generation now probes the modern, namespaced and legacy Mistral SDK import layouts and uses a current model id, fixing breakage against newer SDK releases; calls are throttled and retried with exponential backoff to respect rate limits.
- **Offline data and caching.** The data loader imports `datasets` lazily and reuses cached

JSON splits offline; retrieval indices are cached and keyed by a fingerprint of the corpus so changing the data transparently rebuilds the index.

- **Locked environment.** Dependencies are declared in `pyproject.toml` and pinned in a `uv.lock` (107 packages), so `uv sync` reproduces the exact stack on Python 3.11.

4 Experimental Setup

Data. We evaluate on the first 100 questions of the FairytaleQA test split, shipped as a cached JSON split for fully offline reproduction. The primary human reference answer is used as ground truth.

Metrics. We report Exact Match and token-level F1 (SQuAD-style normalisation), ROUGE-1/2/L (Lin, 2004), BLEU (Papineni et al., 2002), embedding-based semantic similarity, and BERTScore F1 (Zhang et al., 2020), together with mean response time. BERTScore uses `distilbert-base-uncased` to avoid a large model download; the canonical `roberta-large` scorer is configurable.

Configuration. Retrieval $\text{top-}k = 5$, hybrid weight $\alpha = 0.6$, chunk size 200 words. Generation uses temperature 0 and an 80-token cap. All settings live in `config.py` and are overridable via environment variables.

5 Results

Table 1 reports the headline comparison. STARK-QA leads on *every* metric. Its token-F1 of 0.611 is $3.5\times$ Basic-RAG and $14.9\times$ Basic-KAG, and the gap widens on the generation-oriented metrics (BLEU 0.422 vs. 0.085; ROUGE-2 0.467 vs. 0.128), reflecting the value of grounded LLM composition over purely extractive answering. The cost is latency: STARK-QA averages 1.73 s per question (an API round-trip plus proactive throttling) against sub-millisecond local baselines.

Per-type analysis. Table 2 breaks F1 down by question attribute. STARK-QA is strongest on *feeling* (1.00) and *character* (0.82) questions, where the fused entity and emotion evidence is highly predictive, and weakest on *prediction* (0.36) and *outcome resolution* (0.35), which require inference beyond the literal text. The ordering is consistent across systems—every type that is hard for the baselines is also the relative weak point for

Metric	STARK-QA	Basic-RAG	Basic-KAG
Exact Match	0.35	0.00	0.01
Token F1	0.611	0.176	0.041
ROUGE-1	0.629	0.196	0.058
ROUGE-2	0.467	0.128	0.010
ROUGE-L	0.627	0.186	0.055
BLEU	0.422	0.085	0.008
Semantic sim.	0.729	0.381	0.193
BERTScore F1	0.893	0.729	0.657
Avg. time (s)	1.726	0.001	0.003

Table 1: Main results on 100 FairytaleQA test questions. Baselines are deterministic; STARK-QA figures are from the reported Mistral run.

Question type (n)	STARK	B-RAG	B-KAG
feeling (8)	1.00	0.03	0.00
character (9)	0.82	0.10	0.11
causal relationship (19)	0.64	0.26	0.08
action (37)	0.58	0.20	0.01
setting (11)	0.58	0.10	0.03
prediction (7)	0.36	0.13	0.01
outcome resolution (9)	0.35	0.24	0.09

Table 2: Token-F1 by FairytaleQA question type. STARK-QA dominates within every type; the relative ordering of difficulty is shared across systems.

STARK-QA—but STARK-QA dominates within each type, e.g. *feeling* improves from 0.03/0.00 (RAG/KAG) to 1.00.

Discussion. The ablations confirm the design hypothesis. Basic-RAG (lexical-only, extractive) recovers some content on descriptive *causal/action* questions but cannot phrase a clean short answer, capping its F1 near 0.18 and its Exact Match at 0. Basic-KAG is weaker still: co-occurrence neighbourhoods rarely contain the precise answer span. STARK-QA’s combination of dense+sparse retrieval, structured KAG context, and LLM composition lifts both the *retrieval* of relevant evidence and the *surface form* of the answer, which is why the gain is largest on the generation-sensitive metrics (BLEU, ROUGE-2) and on Exact Match.

6 Conclusion

STARK-QA shows that fusing hybrid retrieval with a lightweight knowledge graph and grounding an LLM on the result yields large, consistent gains over either component alone on narrative QA. Equally important for a course/research artifact, the system is now *reproducible*: a single `uv sync` reconstructs the locked environment, cached splits enable offline runs, and graceful fall-

backs keep the pipeline running even when optional dependencies are absent.

Limitations

The evaluation covers 100 test questions from a single split; broader coverage and multiple reference answers would tighten the estimates. STARK-QA’s generation depends on a proprietary LLM, so its scores are not perfectly reproducible run-to-run (the deterministic extractive fallback trades some quality for full reproducibility). Entity and relation extraction in the KAG module is regex-based and English/fairy-tale-specific, and BERTScore uses a lighter encoder than the canonical `roberta-large`, which may shift absolute BERTScore values.

Reproducibility

Code, cached FairytaleQA splits, configuration and the `uv.lock` are included in the repository. To reproduce: `uv sync --python 3.11; uv run python -m spacy download en_core_web_sm; uv run python main.py --test_size 100.` Outputs are written to `results/`.

References

- Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81.
- Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*.
- Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. BLEU: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318.

- Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992.
- Ying Xu, Dakuo Wang, Mo Yu, Daniel Ritchie, Bingsheng Yao, Tongshuang Wu, Zheng Zhang, Toby Jia-Jun Li, Nora Bradford, Branda Sun, Tran Bao Hoang, Yisi Sang, Yufang Hou, Xiaojuan Ma, Diyi Yang, Nanyun Peng, Zhou Yu, and Mark Warschauer. 2022. Fantastic questions and where to find them: FairytaleQA – an authentic dataset for narrative comprehension. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 447–460.
- Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2020. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations (ICLR)*.